

3.4 Terraform remote state

- State is how Terraform knows what it has already built. Sitting on your laptop, it can't be shared with anyone and it can be lost. Every real project uses remote state — ours included.

Learning objective

What is in the state file, and why does Terraform need it?

The Terraform state file (`terraform.tfstate`) is a JSON file that records a snapshot of your managed infrastructure, mapping your configuration code to real-world resources.

It holds all the changes that have been made and the current state of your terraform. This is then used to for things like comparing your changes when you do terraform plan and apply.

What happens if two people run apply at the same time?

If two people run `terraform apply` at the same time, they could both try to change the same infrastructure and update the same state file. This can cause conflicts, incorrect state information, or one person's changes interfering with the other's.

How does an S3 bucket plus a DynamoDB lock table solve that?

The S3 bucket stores the Terraform state in one shared remote location, so everyone uses the same state file instead of having separate copies on their laptops. The DynamoDB lock table prevents two people from changing that state at the same time by creating a lock while one Terraform operation is running. Once the first operation finishes, the lock is released and another person can continue.

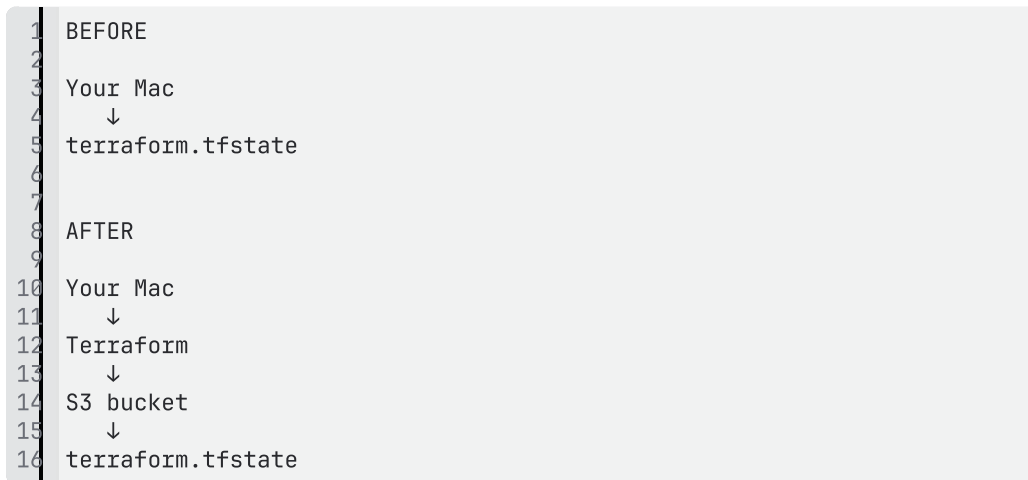
Why can state contain secrets, and why must it never be committed to git?

The sensitive data can become at risk of being seen by anyone.

Task

- Use our own repo — `{{encon-group/terraform-remote-state}}` — to create your S3 bucket and DynamoDB lock table
- Configure your project to use that backend
- Migrate your existing local state to it
- Run a plan afterwards and confirm it shows no changes

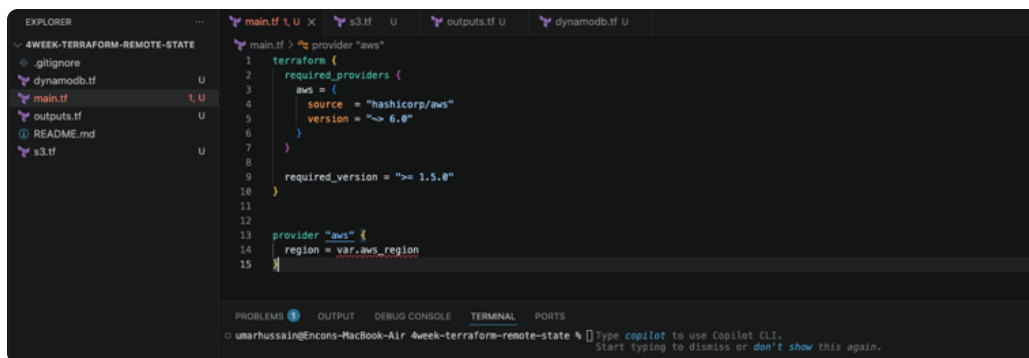
We are going to create an new repo for this as it specifically asks us too in the first built point,
What we're changing:



So the S3 bucket just stores the terraform state file

The git ignore is practically the same as the one for the other part of this project however this time we are not going to gitignore the terraform.lock.hcl.

The image bellow shows that I have created the new repo and cloned it too my VS code; and I have created the new files and structure for this repo.



- gitignore
- README.md
- main.tf
- s3.tf
- Dynamodb.tf
- outputs.tf

What does the [s3.tf](#) file do

The first block:

```
1 resource "aws_s3_bucket" "terraform_state"
```

actually creates the S3 bucket.

The connection:

```
1 bucket = var.state_bucket_name
```

means:

```
1 variables.tf
2 state_bucket_name
3   ↓
4 s3.tf
5 var.state_bucket_name
6   ↓
7 AWS creates that S3 bucket
```

I added:

```
1 prevent_destroy = true
```

because this bucket will eventually contain something extremely important:

```
1 terraform.tfstate
```

We do **not** want an accidental:

```
1 terraform destroy
```

to casually delete your remote state bucket.

Then:

```
1 aws_s3_bucket_versioning
```

keeps previous versions of the state file. If the latest state were accidentally damaged, S3 versioning gives us recovery options.

This block:

```
1 aws_s3_bucket_server_side_encryption_configuration
```

encrypts the state stored in S3, which matters because Terraform state can contain sensitive infrastructure information.

Finally:

```
1 aws_s3_bucket_public_access_block
```

makes sure the state bucket cannot accidentally become public.

So after this file eventually gets applied:

```
1 Terraform
2   ↓
3 S3
4
5 4-week-umar-terraform-remote-state
6   |
7   ├── private
8   ├── encrypted
9   ├── versioned
10  └── protected from accidental Terraform destruction
```

What does the [Dynamodb.tf](#) file do

The important part is:

```
1 hash_key = "LockID"
```

and:

```
1 attribute {
2   name = "LockID"
3   type = "S"
4 }
```

That means the table's primary key is a string called `LockID` .

The flow later will be:

```
1 Person A runs terraform apply
2   ↓
3 Terraform writes a LockID into DynamoDB
4   ↓
5 State is locked
6
7 Person B runs terraform apply
8   ↓
9 Terraform sees the existing lock
10  ↓
11 Second operation cannot safely continue yet
```

Once Person A finishes, Terraform removes the lock.

For this backend repo, `PAY_PER_REQUEST` is a good fit because the table will be used very lightly and you don't need to configure read/write capacity manually.

What does [output.tf](#) file do

These outputs **do not create anything new**. They simply show useful information after Terraform finishes.

For example:

```
1 s3.tf
2 creates bucket
3   ↓
```

```
4 aws_s3_bucket.terraform_state.bucket
5   ↓
6 outputs.tf
7   ↓
8 prints bucket name
```

And:

```
1 dynamodb.tf
2 creates lock table
3   ↓
4 aws_dynamodb_table.terraform_locks.name
5   ↓
6 outputs.tf
7   ↓
8 prints lock table name
```

Then we are going back too the other repo and first. we create a back up using this command:

“cp terraform.tfstate terraform.tfstate.pre-remote-backup”

`.gitignore` already excludes `*.tfstate.*`, so that backup should not be committed. then we create the [backend.tf](#) file and we put the following block off code into this new file:

```
1 terraform {
2   backend "s3" {
3     bucket      = "4-week-umar-terraform-remote-state"
4     key         = "coldchain/terraform.tfstate"
5     region      = "eu-north-1"
6     encrypt     = true
7     dynamodb_table = "4-week-umar-terraform-state-lock"
8   }
9 }
```

A few important things here: the `bucket` is the S3 bucket you just created, `key` is the path where this project's state will live inside that bucket, and `dynamodb_table` tells Terraform which table to use for state locking.

Also, unlike normal Terraform configuration, backend settings **cannot use variables**, so writing the actual bucket and lock-table names directly here is correct.

Now comes the important command. Run:

```
1 terraform init -migrate-state
```

Terraform should detect that you previously had local state and now have an S3 backend. It will ask whether you want to copy the existing state to the new backend.

The flow becomes:

```
1 Existing local terraform.tfstate
2   ↓
3 terraform init -migrate-state
```

```

4           ↓
5 S3 bucket
6           ↓
7 coldchain/terraform.tfstate

```

```

● umarhussain@Encons-MacBook-Air terraform % terraform init -migrate-state
Initializing the backend...
Do you want to copy existing state to the new backend?
  Pre-existing state was found while migrating the previous "local" backend to the
  newly configured "s3" backend. No existing state was found in the newly
  configured "s3" backend. Do you want to copy this state to the new "s3"
  backend? Enter "yes" to copy and "no" to start with an empty state.

  Enter a value: yes

Successfully configured the backend "s3"! Terraform will automatically
use this backend unless the backend configuration changes.

Initializing provider plugins...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Reusing previous version of hashicorp/archive from the dependency lock file
- Using previously-installed hashicorp/aws v6.62.0
- Using previously-installed hashicorp/archive v2.8.0

Warning: Deprecated Parameter

  on backend.tf line 7, in terraform:
   7:     dynamodb_table = "4-week-umar-terraform-state-lock"

The parameter "dynamodb_table" is deprecated. Use parameter "use_lockfile" instead.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.

```

After this i sent terraform plan and the result I needed “was no changes had been made” if there had been then something had gone wrong. after running it, it showed no changes had been made so migration had been successful.

```

aws_iam_role_policy.excursion_handler.access: Refreshing state... [id=umar-coldchain-excursion-han
aws_iam_role_policy.attachment.link_dynamodb_logs: Refreshing state... [id=umar-coldchain-LinkDynam
aws_iam_role_policy.link_dynamodb.access: Refreshing state... [id=umar-coldchain-LinkDynamoDB-role-
aws_lambda_function.excursion_handler: Refreshing state... [id=umar-coldchain-excursion-handler-te
aws_lambda_function.link_dynamodb: Refreshing state... [id=umar-coldchain-LinkDynamoDB-terraform]
aws_lambda_event_source_mapping.alert_queue_to_handler: Refreshing state... [id=59d202d4-8881-4211-
aws_lambda_permission.allow_api_gateway: Refreshing state... [id=AllowRESTAPIGatewayInvoke]
aws_api_gateway_integration.patch_reading: Refreshing state... [id=agi-np6w972j8k-6rpzxo-PATCH]
aws_api_gateway_integration.delete_reading: Refreshing state... [id=agi-np6w972j8k-6rpzxo-DELETE]
aws_api_gateway_integration.get_one_reading: Refreshing state... [id=agi-np6w972j8k-6rpzxo-GET]
aws_api_gateway_integration.get_readings: Refreshing state... [id=agi-np6w972j8k-lnubce-GET]
aws_api_gateway_integration.post_reading: Refreshing state... [id=agi-np6w972j8k-lnubce-POST]
aws_api_gateway_deployment.coldchain: Refreshing state... [id=gz4L2a]
aws_api_gateway_stage.dev: Refreshing state... [id=ags-np6w972j8k-dev]
aws_api_gateway_usage_plan.coldchain: Refreshing state... [id=e0raxx]
aws_api_gateway_usage_plan_key.coldchain: Refreshing state... [id=ynr7pydtw1]

No changes. Your infrastructure matches the configuration.

```

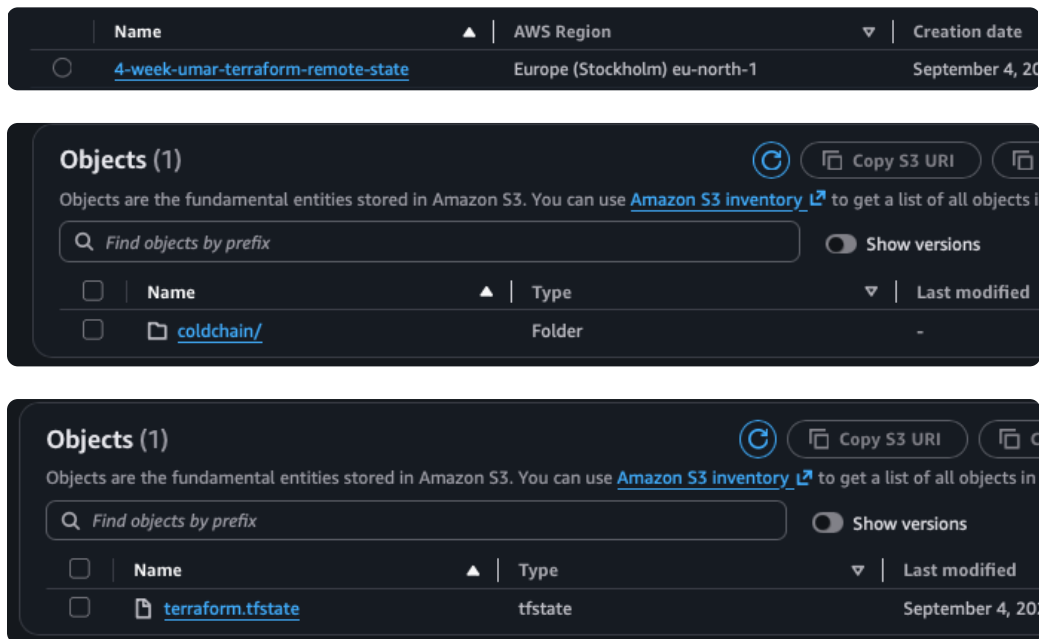
So with this working what have we accomplished:

- ✓ Created remote-state repository
- ✓ Created S3 state bucket
- ✓ Enabled S3 versioning
- ✓ Enabled encryption

- ✓ Blocked public access
- ✓ Created DynamoDB LockID table
- ✓ Added [backend.tf](#) to cold-chain project
- ✓ Migrated local state → S3
- ✓ terraform plan after migration
- ✓ No unexpected infrastructure changes

(terraform apply DIDN'T NEED to be run this is because there was no changes were made to any of the Infrastructure or nothing needed to be created. we just need to see it work that why he plan was ran).

Testing



The above images show that the s3 worked and was configured properly.

The image above shows that the database lock is working because two apply cannot run at the same time. (I couldn't actually get an image because the apply was running too fast because there was nothing to change so I couldn't run two at once. I know it works just have to trust me)